

5-22 上位机通信协议三方核对：PDF 文档 / 当前代码 / 实际日志

生成时间：2026-05-25 10:20:53 EEST

核对目标：把 docs 中 PDF 协议、当前代码实现、5-22 日志实际收发帧放在一起对比，找出上位机协议排查时最容易出错的地方。

1. 核对范围

类型	路径 / 来源	作用
PDF 协议文档	/home/oneko/projects/spray_b2w_robot_project_greek/docs/喷涂机器人通信协议03-10.pdf	上位机侧可能依据的正式协议文档
docx 协议文档	/home/oneko/projects/spray_b2w_robot_project_greek/app_ws/喷涂机器人通信协议03-10.docx	旧/副本协议；本次只作为补充参考，主对比以 PDF 为准
TCP 代码入口	/home/oneko/projects/spray_b2w_robot_project_greek/app_ws/src/app_node.cpp	TCP 9002 服务端、收包解析、机器人主动上报封包
下游执行代码	/home/oneko/projects/spray_b2w_robot_project_greek/b2w_navigation_ws/src/b2w_teleop.cpp	/joy 到 Unitree SportClient 的动作映射
5-22 日志	/home/oneko/projects/spray_b2w_robot_project_greek/dog_logs/5-22/5-22/log	实际收发帧与现场统计

代码证据主要来自：

- app_node.cpp:265-360：parseNextPacket()，APP -> 机器人收包解析。
- app_node.cpp:365-469：handleCommandPacket()，控制子命令解析。
- app_node.cpp:471-485：handleRegionPacket()，区域包解析。
- app_node.cpp:1285-1457：sendPointCloud()、sendAllPoints()、sendProgress()、sendPosition()、sendMaxTemperature()、sendPath()。
- b2w_teleop.cpp:41-76：/joy 到 StandDown/StandUp/Move/StopMove。

2. 一页结论：最需要和上位机确认的差异

优先级	功能	PDF 写法	当前代码实际	5-22 日志实际	结论 / 风险
高	0xFF	PDF 同时写成心跳，又在控制列表写成“急停 Damp”	代码只把 top-level 0xFF 当心跳，不执行 Damp	有心跳状态日志 15 次、超时 36 次；没有 RX COMMAND ... FF ... 执行动作	上位机不要把 0xFF 当急停；如果发 F5 08 01 00 FF 00 00 5F 只会变未知子命令
高	0x06 进度	上传表漏了 2 字节长度，看起来像 F5 06 PP 00 00 5F	实际 F5 06 01 00 PP 00 00 5F，8 字节	TX PROGRESS [len=8] 226 次，值 0~100	上位机必须按 8 字节解析，并把 PP 当 0~100 百分比
高	0x09 区域	每区域像 32 字节，指令类型 2 字节	每区域按 64 字节跳过，指令类型 1 字节	5-22 没有 RX REGION	上位机若按 PDF 发区域包，机器人很可能等包/错位

优先级	功能	PDF 写法	当前代码实际	5-22 日志实际	结论 / 风险
中	0x04 轨迹	表格写 24 字节/点，但文字写 float 12 字节/点	实际 3 个 double，24 字节/点	仅日志 Sent Path with N points 942 次，不打印十六进制	上位机必须按 double 解析，不能按 float
中	通用格式	PDF 第 1 节写所有包都有 2 字节数据段长度	只有部分包有长度；0x04/0x05/0x07/0x09 没有通用长度字段	0x07 日志固定 len=29；0x05 固定 6 字节但不打印 hex	上位机解析必须按功能码分支，不能套一个通用 length parser
中	CRC	PDF 所有包都有 CRC	代码保留字段但发送固定 00 00，接收不校验	日志中实际帧 CRC 都是 00 00	当前 CRC 只是占位，不能用来判断误码

3. 5-22 日志总体统计

扫描范围：76 个 robot_tcp.log。

项	统计
TCP client 连接	95 次
连接来源 IP	全部为 192.168.88.27
RX COMMAND 控制帧	7387 条
RX REGION 区域帧	0 条
TX ALL_POINTS 全部点位	95 条
TX PROGRESS 进度	226 条
TX POSITION 位置	80012 条
Sent max temperature 温度	2815 条
Sent Path 轨迹	942 条
Sent PointCloud 点云	998 条
APP heartbeat state changed	15 条
APP heartbeat timeout	36 条
Unsupported function code	0 条
Invalid tail	0 条
Discarding invalid ...	0 条

控制子命令统计：

子命令	含义	日志次数
0x01	start / 开始喷涂	72
0x02	pause / 暂停	79
0x03	stop / 停止	63
0x04	前进	5810
0x05	后退	377
0x06	左移	7
0x07	右移	10
0x08	左转	435
0x09	右转	410

子命令	含义	日志次数
0x0A	StandDown / 趴下	52
0x0B	StandUp / 站立	46
0x10	resume / 恢复	26

进度值统计：TX PROGRESS 共 226 条，最小 0，最大 100。

温度值统计：Sent max temperature 共 2815 条，最小 26，最大 49。

4. 总体帧格式核对

PDF 第 1 节写通用格式：

```
F5 功能码 数据段长度(2字节) 数据段 CRC低 CRC高 5F
```

当前代码不是所有包都符合这个统一格式。

功能码	方向	当前代码实际格式	是否有 2 字节长度 字段	说明
0x08	APP -> 机器人	F5 08 len_low len_high cmd [extra...] 00 00 5F	有	和 PDF 控制协议基本一致
0xFF	APP -> 机器人	F5 FF 01 00 FF 00 00 5F	有	心跳包，1s 一次；不执行动作
0x09	APP -> 机器人	F5 09 region_count [region*64] instruction 00 00 5F	无	代码按区域数量推总长
0x01	机器人 -> APP	F5 01 count_low count_high [point*24] 00 00 5F	不是字节长度，是 点数	全部点位
0x02	机器人 -> APP	F5 02 count_low count_high [point*24] 00 00 5F	不是字节长度，是 点数	已完成点云
0x03	机器人 -> APP	F5 03 count_low count_high [point*24] 00 00 5F	不是字节长度，是 点数	未完成点云
0x04	机器人 -> APP	F5 04 N [point*24] 00 00 5F	无	N 是 1 字节点数
0x05	机器人 -> APP	F5 05 TT 00 00 5F	无	TT 是最高温度
0x06	机器人 -> APP	F5 06 01 00 PP 00 00 5F	有	PP 是 0~100 进度
0x07	机器人 -> APP	F5 07 [x double][y double][z double] 00 00 5F	无	固定 29 字节

结论：上位机解析必须先看功能码，再按功能码分支解析。不能对所有机器人上报包统一读 len_low/len_high。

5. APP -> 机器人：下发协议对比

5.1 心跳 func_code=0xFF

PDF 写法：

```
字段：包头 功能码 数据段长度 心跳值 CRC低 CRC高 包尾
值：   F5   FF      2字节      FF      1字节 1字节 5F
示例：F5 FF 01 00 FF 00 00 5F
```

当前代码实际：一致，但语义必须强调：这是 top-level 功能码 0xFF，不是 0x08 控制帧里的子命令。

实际格式：

```
F5 FF 01 00 FF 00 00 5F
```

代码行为：

- data_len 必须等于 1。
- payload 必须等于 0xFF。
- 只更新心跳状态。
- 不发布 /joy。
- 不执行 Damp()。
- 稳态心跳不逐帧打印，只打印状态变化。

代码证据：app_node.cpp:309-336, app_node.cpp:494-528。

5-22 日志证据：

```
APP heartbeat state changed: online (control_session_active=false)
APP heartbeat timeout 3.69s > 3.50s; entering disconnect protection.
```

5-22 统计：

- APP heartbeat state changed：15 次。
- APP heartbeat timeout：36 次。

重要冲突：PDF 控制指令列表又写：

```
0xFF : 急停 int32_t Damp()
```

这和当前代码冲突。当前代码已经不是 0xFF -> Damp()。如果上位机发：

```
F5 08 01 00 FF 00 00 5F
```

机器人会把它当作 0x08 控制帧中的未知 instruction，结果不是心跳，也不是 Damp。

5.2 控制帧 func_code=0x08

PDF 写法：

```
F5 08 数据段长度(2字节) 指令类型(1字节) CRC低 CRC高 5F
```

当前代码实际：一致。5-22 实际控制帧全部是 8 字节：

```
F5 08 01 00 XX 00 00 5F
```

字段说明：

F5	包头
08	功能码：控制帧
01 00	数据段长度，小端，payload 长度 1 字节
XX	指令类型

00 00CRC 占位, 当前不校验
5F包尾

5-22 日志样本:

RX COMMAND [len=8] : F5 08 01 00 04 00 00 5F
RX COMMAND [len=8] : F5 08 01 00 0A 00 00 5F

当前代码支持的指令:

子命令	PDF	代码实际动作	下游执行	5-22 次数
0x01	开始 Start	启动 /home/test/start_all.sh; 发布 /remote_command=start	启动任务	72
0x02	暂停 Pause	handlePauseCommand(); 调用 /emergency_stop	导航 emergency pause	79
0x03	结束 STOP	handleStopCommand(); StopMove + Z1 reset + kill start_all	停止任务	63
0x04	前进	发布 /joy.axes=[0,+1,0]	Move(linear_x>0)	5810
0x05	后退	发布 /joy.axes=[0,-1,0]	Move(linear_x<0)	377
0x06	左移	发布 /joy.axes=[+1,0,0]	Move(linear_y>0)	7
0x07	右移	发布 /joy.axes=[-1,0,0]	Move(linear_y<0)	10
0x08	左转	发布 /joy.axes=[0,0,+1]	Move(yaw>0)	435
0x09	右转	发布 /joy.axes=[0,0,-1]	Move(yaw<0)	410
0x0A	趴下 StandDown	发布 /joy.buttons[0]=1	SportClient::StandDown()	52
0x0B	站立 StandUp	发布 /joy.buttons[1]=1	SportClient::StandUp()	46
0x10	暂停后恢复 Restart	调用 /erase_emergency_stop	解除 emergency pause	26
0xFF	PDF 写急停 Damp	当前代码不支持为控制子命令	unknown instruction	0

代码证据: app_node.cpp:365-469, b2w_teleop.cpp:41-76。

5.3 区域帧 func_code=0x09

PDF 写法:

F5 09 区域数量 边框点1 边框点2 边框点3 边框点4 指令类型 CRC低 CRC高 5F

PDF 字段宽度:

- 区域数量: 1 字节。
- 边框点 1~4: 每个 8 字节。
- 指令类型: 2 字节。
- 0x11: 优先区域。
- 0x12: 避让区域。

当前代码实际解析:

F5 09 region_count [region_count * 64 bytes] instruction_type 00 00 5F

代码规则：

- `region_count = packet[2]`。
- 总长 = $1 + 1 + 1 + \text{region_count} * 64 + 1 + 2 + 1$ 。
- `instruction_type = packet[3 + \text{region\_count} * 64]`，只有 1 字节。
- 收到后只打印 priority/avoidance 日志，没有实际下发给导航/规划。

代码证据：app_node.cpp:339-355, app_node.cpp:471-485。

5-22 日志：没有 RX REGION。

结论：区域包当前是高风险不一致项。PDF 像是每区域 32 字节 + 2 字节指令类型；代码是每区域 64 字节 + 1 字节指令类型。若上位机按 PDF 发，机器人解析大概率错位或等待更多字节。

6. 机器人 -> APP：主动上传协议对比

6.1 0x01 全部点位 ALL_POINTS

PDF 写法：

```
F5 01 点数量N(2字节) 目标点1(24字节) ... 目标点N(24字节) CRC低 CRC高 5F
```

当前代码实际：一致。

实际格式：

```
F5 01 count_low count_high [x double][y double][z double]... 00 00 5F
```

字段：

- `count_low count_high` 是 `uint16_t` 小端点数，不是字节长度。
- 每点 24 字节：x/y/z 各 1 个 little-endian double。
- 数据源：`$HOME/gnss_waypoints.txt`，格式 `id, x, y, z`。
- 新 client 连接后立即发送。

代码证据：app_node.cpp:1327-1357。

5-22 日志统计：TX ALL_POINTS 95 次。

5-22 样本：

```
New client connected from 192.168.88.27
TX ALL_POINTS [len=2431] : F5 01 65 00 ... 00 00 5F
```

样本解释：

- `65 00` = 101 个点。
- 总长度 $2431 = 1 + 1 + 2 + 101 * 24 + 2 + 1$ 。

6.2 0x02 已完成点云 ACQUIRED_POINTS

PDF 写法：

```
F5 02 点数量N(2字节) 目标点1(24字节) ... 目标点N(24字节) CRC低 CRC高 5F
```

当前代码实际：一致。

实际格式：

```
F5 02 count_low count_high [x double][y double][z double]... 00 00 5F
```

字段:

- `count_low count_high` 是 `uint16_t` 小端点数。
- 每点 24 字节, 3 个 double。
- 从 `/acquired_points` 的 `PointCloud2` 中读取 float x/y/z, 然后转 double 发送。

代码证据: `app_node.cpp:106-109`, `app_node.cpp:1285-1325`。

5-22 日志样本:

```
Sent PointCloud: 1 points func=0x02  
Sent PointCloud: 2 points func=0x02
```

注意: 当前代码没有对 `0x02` 打印十六进制 `TX ...`, 日志只打印点数和功能码。

6.3 `0x03` 未完成点云 UNACQUIRED_POINTS

PDF 写法:

```
F5 03 点数量N(2字节) 目标点1(24字节) ... 目标点N(24字节) CRC低 CRC高 5F
```

当前代码实际: 一致。

实际格式:

```
F5 03 count_low count_high [x double][y double][z double]... 00 00 5F
```

字段:

- `count_low count_high` 是 `uint16_t` 小端点数。
- 每点 24 字节, 3 个 double。
- 从 `/unacquired_points` 的 `PointCloud2` 中读取 float x/y/z, 然后转 double 发送。

代码证据: `app_node.cpp:111-114`, `app_node.cpp:1285-1325`。

5-22 日志样本:

```
Sent PointCloud: 99 points func=0x03
```

注意: 当前代码没有对 `0x03` 打印十六进制 `TX ...`, 日志只打印点数和功能码。

6.4 `0x04` 机器人轨迹 PATH

PDF 写法:

```
F5 04 点数量N(1字节) 轨迹点1(24字节) ... 轨迹点N(24字节) CRC低 CRC高 5F
```

PDF 内部矛盾:

- 表格写 `24字节/点`。
- 文字又写“X、Y、Z 三个浮点数, 各占 4 字节, 共 12 字节/点”。

当前代码实际以表格为准: 每点 24 字节。

实际格式：

```
F5 04 N [x double][y double][z double]... 00 00 5F
```

字段：

- 没有 2 字节长度字段。
- **N** 是 1 字节点数，最多 255。
- 每点 24 字节：3 个 little-endian double。
- 当路径超过 255 点时截断为 255。

代码证据：app_node.cpp:1413-1457。

5-22 日志统计：Sent Path 942 次。

5-22 日志样本：

```
Sent Path with 1 points
Sent Path with 7 points
Sent Path with 31 points
Sent Path with 255 points
```

注意：当前代码没有对 0x04 打印十六进制 TX PATH，日志只打印点数。

6.5 0x05 当前电机最高温度

PDF 写法：

```
F5 05 温度值(1字节) CRC低 CRC高 5F
```

当前代码实际：一致。

实际格式：

```
F5 05 TT 00 00 5F
```

字段：

- 总长固定 6 字节。
- 没有 2 字节长度字段。
- **TT** 是所有电机温度中的最大值，不是温度数组。
- 代码把最大温度 clamp 到 0~255 后转 uint8_t。

代码证据：app_node.cpp:98-101, app_node.cpp:1394-1411。

5-22 日志统计：Sent max temperature 2815 次，最小 26，最大 49。

5-22 日志样本：

```
Sent max temperature: 42
Sent max temperature: 44
```

对应实际 TCP 包示例：

```
42°C -> F5 05 2A 00 00 5F
```

```
44°C -> F5 05 2C 00 00 5F
```

注意：当前代码没有对 0x05 打印十六进制 TX TEMPERATURE，日志只打印十进制温度值。

6.6 0x06 任务进度 PROGRESS

PDF 写法：

```
F5 06 进度值(1字节) CRC低 CRC高 5F
```

PDF 问题：上传进度表漏了 2 字节数据段长度。

当前代码实际：

```
F5 06 01 00 PP 00 00 5F
```

字段：

- 总长固定 8 字节。
- 01 00 是 payload 长度，表示后面进度值 1 字节。
- PP 是 0~100 百分比，不是 0~255 比例值。
- CRC 固定 00 00。

代码证据：app_node.cpp:89-93, app_node.cpp:1359-1372。

5-22 日志统计：TX PROGRESS 226 次，最小 0，最大 100。

5-22 日志样本：

```
TX PROGRESS [len=8] : F5 06 01 00 00 00 00 5F
TX PROGRESS [len=8] : F5 06 01 00 01 00 00 5F
TX PROGRESS [len=8] : F5 06 01 00 60 00 00 5F
TX PROGRESS [len=8] : F5 06 01 00 64 00 00 5F
```

解释：

- 00 = 0%。
- 01 = 1%。
- 60 = 96%。
- 64 = 100%。

6.7 0x07 当前坐标位置 POSITION

PDF 写法：

```
F5 07 X轴坐标(8字节) Y轴坐标(8字节) Z轴坐标(8字节) CRC低 CRC高 5F
```

当前代码实际：一致。

实际格式：

```
F5 07 [x double][y double][z double] 00 00 5F
```

字段：

- 总长固定 29 字节。

- 没有 2 字节长度字段。
- x/y/z 各 8 字节 little-endian double。
- 数据源：/b2w_odom。

代码证据：app_node.cpp:94-97, app_node.cpp:1374-1393。

5-22 日志统计：TX POSITION 80012 次。

5-22 日志样本：

```
TX POSITION [len=29] : F5 07 D1 BD E4 4B 7C 64 1D 41 0B 9B 56 94 A0 0F 50 41 F2 41 CF 66 D5 29 67
40 00 00 5F
```

注意：如果上位机按“通用格式”把 0x07 后面的两个字节当 len_low/len_high，会把 x 坐标 double 的前两个字节误当长度，后续全部错位。

7. 日志实际发送/接收样本汇总

7.1 APP -> 机器人实际收到的帧

控制帧：

```
RX COMMAND [len=8] : F5 08 01 00 04 00 00 5F
RX COMMAND [len=8] : F5 08 01 00 0A 00 00 5F
```

心跳帧：

- 稳态心跳不逐帧打印十六进制。
- 只打印状态变化或超时。

```
APP heartbeat state changed: online (control_session_active=false)
APP heartbeat timeout 3.69s > 3.50s; entering disconnect protection.
```

区域帧：

5-22 未发现 RX REGION

7.2 机器人 -> APP 实际发送的帧

全部点位 0x01：

```
TX ALL_POINTS [len=2431] : F5 01 65 00 ... 00 00 5F
```

进度 0x06：

```
TX PROGRESS [len=8] : F5 06 01 00 00 00 00 5F
TX PROGRESS [len=8] : F5 06 01 00 64 00 00 5F
```

位置 0x07：

```
TX POSITION [len=29] : F5 07 ...24字节double坐标... 00 00 5F
```

温度 0x05:

Sent max temperature: 42

实际对应:

F5 05 2A 00 00 5F

轨迹 0x04:

Sent Path with 255 points

实际对应:

F5 04 FF [255 * 24字节节点数据] 00 00 5F

点云 0x02/0x03:

Sent PointCloud: 1 points func=0x02
Sent PointCloud: 99 points func=0x03

实际对应:

F5 02 01 00 [1 * 24字节节点数据] 00 00 5F
F5 03 63 00 [99 * 24字节节点数据] 00 00 5F

8. 上位机解析建议

8.1 必须按功能码分支解析

建议上位机解析逻辑:

```
if func == 0x01/0x02/0x03:
    read uint16 little-endian count
    read count * 24 bytes of double x/y/z
elif func == 0x04:
    read uint8 count
    read count * 24 bytes of double x/y/z
elif func == 0x05:
    read uint8 max_temperature
elif func == 0x06:
    read uint16 payload_len, must be 1
    read uint8 progress_percent
elif func == 0x07:
    read exactly 24 bytes as 3 doubles
elif func == 0x08:
    APP -> robot only; read uint16 payload_len, first payload byte is instruction
elif func == 0x09:
    当前代码按 region_count * 64 + 1 解析, 但 PDF 与代码冲突, 需双方重新确认
elif func == 0xFF:
    APP -> robot heartbeat only: F5 FF 01 00 FF 00 00 5F
```

8.2 不要按 PDF 通用格式解析所有上报包

高风险错误：

```
F5 07 xx xx ...
```

如果把 `xx xx` 当 length，位置包会立刻错位。

同理：

- `0x05` 的温度值会被误当 length low。
- `0x04` 的点数量 N 会被误当 length low。

8.3 坐标全部按 double 解析

涉及坐标的功能码：`0x01/0x02/0x03/0x04/0x07`。

统一解析为：

```
x: little-endian double, 8 字节  
y: little-endian double, 8 字节  
z: little-endian double, 8 字节
```

不要按 float 解析。

8.4 当前 CRC 不参与校验

现状：

- 机器人发送固定 `00 00`。
- 机器人接收不校验 CRC。
- 上位机若实现 CRC 校验，当前会因为双方没有真实 CRC 算法而不兼容。

如果要启用 CRC，需要双方先明确 CRC 算法、覆盖范围、大小端，再同步修改。

9. 排查协议问题时优先抓这几类证据

1. 上位机侧打印原始十六进制帧，至少包括：
 - 发送给机器人：`0x08` 控制、`0xFF` 心跳、`0x09` 区域。
 - 从机器人收到：`0x01~0x07` 上报。
2. 机器人侧对齐 `robot_tcp.log`：
 - `RX COMMAND`
 - `RX REGION`
 - `TX ALL_POINTS`
 - `TX PROGRESS`
 - `TX POSITION`
 - `Sent max temperature`
 - `Sent Path`
 - `Sent PointCloud`
 - `APP heartbeat state changed`
 - `APP heartbeat timeout`
3. 如果排查“趴下/急停”：
 - 查 `RX COMMAND ... 0A ...`。
 - 查 `Published Joy action for instruction 0x0A`。
 - 查 `b2w_teleop.log` 中 `Executed StandDown from APP command.`。
 - 不要把 `0xFF` 心跳误判为急停。
4. 如果排查“进度显示异常”：
 - 确认上位机按 8 字节解析 `F5 06 01 00 PP 00 00 5F`。
 - 确认 PP 直接按 0~100 显示，不要按 255 缩放。
5. 如果排查“位置/轨迹坐标错乱”：

- 确认 0x07 没有 length, 固定 29 字节。
 - 确认 0x04 每点 24 字节 double, 不是 12 字节 float。
6. 如果排查“区域包无响应”:
- 先暂停按 PDF 实现区域包。
 - 需要和机器人代码统一每区域字节数到底是 32 还是 64、指令类型是 1 字节还是 2 字节。

10. 快速 grep 命令

```
# 机器人 TCP 控制/上报/心跳
grep -RInE "New client|RX COMMAND|RX REGION|TX ALL_POINTS|TX PROGRESS|TX POSITION|Sent max
temperature|Sent Path|Sent PointCloud|APP heartbeat|Unsupported function|Invalid tail|Discarding
invalid" \
/home/oneko/projects/spray_b2w_robot_project_greek/dog_logs/5-22/5-22/log/**/robot_tcp.log

# 姿态执行确认
grep -RInE "Executed StandDown|Executed StandUp|SportClient" \
/home/oneko/projects/spray_b2w_robot_project_greek/dog_logs/5-22/5-22/log/**/b2w_teleop.log

# 最新一次日志重点事件
grep -nE "New client|RX COMMAND|RX REGION|TX PROGRESS|TX POSITION|Sent max temperature|Sent
Path|APP heartbeat|Client disconnected|StopMove|Emergency stop" \
/home/oneko/projects/spray_b2w_robot_project_greek/dog_logs/5-22/5-
22/log/tcp_base_ctl_latest/robot_tcp.log
```

11. 最终建议

当前最应该和上位机同步修正/确认的协议点:

1. 0xFF 只作为 top-level 心跳: F5 FF 01 00 FF 00 00 5F; 不要再作为急停/Damp 控制命令。
2. 0x06 进度实际为 8 字节: F5 06 01 00 PP 00 00 5F; PP 是 0~100。
3. 0x04/0x05/0x07 没有 2 字节 length 字段。
4. 所有坐标点按 little-endian double 解包, 每点 24 字节。
5. 0x09 区域包 PDF 与代码不一致, 必须单独对齐后再联调。
6. CRC 当前只是 00 00 占位, 不要单方面启用 CRC 校验。